Lesson 10: Play It Again

Unity

VIDEO GAME CREATION

This week: when the round ends, the game actually stops - and pressing R brings it back to a genuinely fresh start.

Open your own [YourName]Game project from Lesson 9 - don't start a new one.

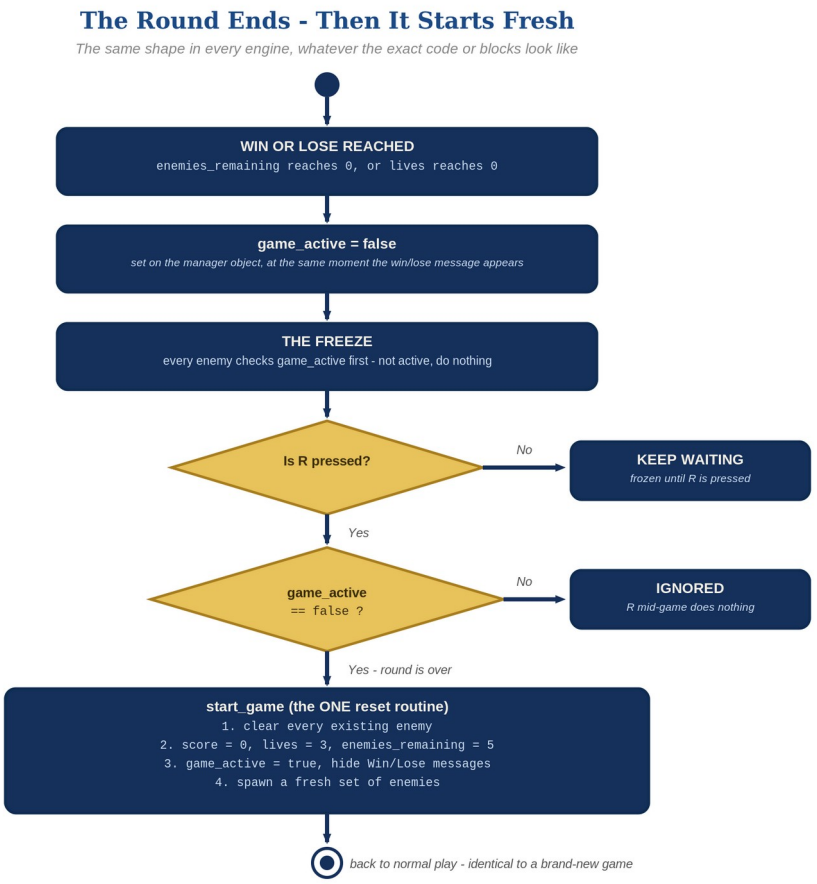
Tip: no new assets this week - this is a logic lesson, and it reuses your very first setup code from Lesson 5.

Setup

1. Open your [YourName]Game project from Lesson 9.
2. Select your enemy prefab and add a new Tag named "Enemy" (Inspector > Tag dropdown > Add Tag), then assign it to the prefab.

Level 1 - Minimal (MVP)

When the round ends (win or lose), the game freezes - score and lives stop changing, and further enemy hits/escapes do nothing. Pressing R resets everything and starts a new round.



When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.

1. In GameManager.cs, add public static bool gameActive = true, and update RegisterKill() and RegisterEscape() so that when hasWon or hasLost becomes true, gameActive is set to false too (code below).
2. In EnemySpawner.cs, split your existing spawn code into two public methods: ClearEnemies() (destroys every GameObject tagged "Enemy") and SpawnEnemies() (your existing spawn loop), and call SpawnEnemies() from Start() (code below).
3. In GameManager.cs, add a public EnemySpawner spawner field, and a public method ResetGame() (not static) that calls spawner.ClearEnemies(), resets score/lives/enemiesRemaining/hasWon/hasLost/gameActive, then calls spawner.SpawnEnemies(). Back in Unity, select the GameManager GameObject and drag the object that has EnemySpawner.cs from the Hierarchy onto its new Spawner slot in the Inspector, the same way you assigned the enemy prefab in Lesson 5.
4. In Enemy.cs's Update(), add "if (!GameManager.gameActive) return;" as the very first line.
5. In GameManager.cs, add an Update() that checks for Keyboard.current.rKey.wasPressedThisFrame and calls ResetGame() if the game isn't active (code below). If this is the first Input System check in GameManager.cs, add

using UnityEngine.InputSystem; at the top of the file alongside using UnityEngine;. Update the "You Win!" and "Game Over" labels in OnGUI() to include "Press R to Restart".

6. Press Play, play to a win or loss, confirm movement freezes, then press R and confirm everything resets and play resumes.

Highlighted lines are new this week; everything else is code you already have.

```
// EnemySpawner.cs (complete):
public GameObject enemyPrefab;
public int enemyCount = 5;

void Start() {
    SpawnEnemies();
}

public void ClearEnemies() {
    foreach (GameObject oldEnemy in
GameObject.FindGameObjectsWithTag("Enemy")) {
        Destroy(oldEnemy);
    }
}

public void SpawnEnemies() {
    for (int enemyIndex = 0; enemyIndex < enemyCount; enemyIndex++) {
        Vector3 pos = new Vector3(-4f + enemyIndex * 2f, 4f, 0f);
        Instantiate(enemyPrefab, pos, Quaternion.identity);
    }
}

// GameManager.cs (the new and revised members):
using UnityEngine.InputSystem;

public static bool gameActive = true;

public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    if (enemiesRemaining <= 0) {
        hasWon = true;
        gameActive = false;
    }
}

public static void RegisterEscape() {
    lives--;
    if (lives <= 0) {
        hasLost = true;
        gameActive = false;
    }
}

public EnemySpawner spawner;

public void ResetGame() {
    spawner.ClearEnemies();
```

```
    score = 0;
    lives = 3;
    enemiesRemaining = 5;
    hasWon = false;
    hasLost = false;
    gameActive = true;
    spawner.SpawnEnemies();
}

void Update() {
    if (!gameActive && Keyboard.current.rKey.wasPressedThisFrame) {
        ResetGame();
    }
}

// Enemy.cs - add as the very first line of Update():
void Update() {
    if (!GameManager.gameActive) return;
    // ...existing movement and escape code...
}
```

Alternate Path: Visual Scripting

The same reusable-reset pattern works in a Script Graph instead of C#, using the same new Input System as the code path.

1. Open PlayerControls (the Input Actions asset from Lesson 2's Player Input component) and add a new action to the Player action map named Restart, Action Type Button, bound to R. Click Save Asset.
2. Build a Custom Event graph named "ResetGame" on a persistent GameObject, containing nodes that clear existing enemy objects, reset your score/lives/enemies variables, and re-trigger your spawn graph.
3. On a graph on the Player (or PlayerInput) GameObject, right-click and add an "On Input System Event Button" node. Leave Target as This, set Input Action to Restart, and set Input Action Change Type to On Pressed. Wire its Trigger output into a Branch node checking GameActive is false, then into the ResetGame custom event on True.

An On Input System Event Button node set to On Pressed only fires once per physical press (event-driven, not polled), same as wasPressedThisFrame in the code path - no extra debounce logic needed.

Before You Test: Audit Your Game State

Game state is everything your game remembers right now: score, lives, positions, costumes and colors, animations, sounds, and messages on screen. Until this lesson, your game started once and ended once, so nothing ever had to be put back. A restart means ALL of it must go back to how it was at the start - including anything extra you added that this handout doesn't mention.

1. Find EVERY Awake() and Start() in every script. Both run only once, when the object first loads, so they will NOT run again on restart. Anything they set up must also be reset by ResetGame().

2. Remember static fields (like GameManager's score and lives) keep their value across restarts until you set them. Instance fields on other scripts (the player's speed, an enemy counter) are also state.
3. Look at everything that happens at game over: Animator states or triggers, color or sprite changes, Time.timeScale, coroutines you started, particle effects, AudioSources, objects you turned off with SetActive(false), UI text.
4. For each, add the opposite to ResetGame(): reset the Animator, restore the color and sprite, set Time.timeScale back to 1, stop coroutines, stop the particles and audio, SetActive(true) where needed, and put the player back at its starting position.

Rule of thumb: if it changed when the round ended (or at any point during play), something has to change it back when the round restarts.

Level 2 - Complete (Alpha/Beta)

A restart is a genuinely clean slate - no leftover enemies, no stale score/lives, and it works correctly no matter how many times in a row you play.

- Play to a win, confirm movement freezes, then press R and confirm score/lives/enemiesRemaining are exactly back to 0/3/5 with no leftover enemy GameObjects.
- Repeat for a loss - same clean reset.
- Press R while the game is still active and confirm NOTHING happens.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Fix a real bug! We laid the groundwork for this bug all the way back in Lesson 6 when we defined our win scenario. Then in Lesson 8 we defined our lose scenario. By thinking in terms of win and lose separately, rather than in terms of wave over and game over, we created a bug. Find it and resolve it in the manner you, the Designer, think best. (Hint: play a round where you destroy some enemies and let the others escape, and watch what happens to the round, and to your R key.)
- Visual: add a fade or scale animation to the win/lose text instead of it just appearing instantly.
- Structure: notice ResetGame() is now the single place that sets up a fresh game - called both from Start() (via SpawnEnemies(), indirectly the first time) and directly on restart.

If Something's Wrong

- NullReferenceException inside ResetGame(): the Spawner slot on the GameManager component is empty. Select the GameManager GameObject and drag the object that has EnemySpawner.cs onto the Spawner slot.
- Restart leaves old enemies on screen: confirm the enemy prefab actually has the "Enemy" tag assigned - a forgotten tag means FindGameObjectsWithTag() finds nothing.
- Enemies still move after game-over: confirm "if (!GameManager.gameActive) return;" is the very FIRST line inside Update(), before any movement code runs.
- R does nothing: confirm GameManager.cs is attached to an active GameObject in the scene (its Update() only runs if it is), and that using UnityEngine.InputSystem; is at the top of the file.

Want More?

Student Resource List > Unity section, or learn.unity.com